# ELM-FBPINN: Efficient Finite-Basis Physics-Informed Neural Networks

Samuel Anderson, Victorita Dolean, Ben Moseley, and Jennifer Pestana

## 1 Introduction

Solving partial differential equations (PDEs) efficiently and accurately is crucial for scientific research. Traditional numerical methods like finite difference (FDM) and finite element methods (FEM) face fundamental trade-offs between accuracy and efficiency. Scientific machine learning has introduced alternative approaches, including physics-informed neural networks (PINNs), which solve boundary value problems of the form:

$$\mathcal{N}[u(x)] = f(x), \quad x \in \Omega \subset \mathbb{R}^d, \tag{1}$$

$$\mathcal{B}_k[u(x)] = g_k(x), \quad x \in \Gamma_k \subset \partial\Omega, \tag{2}$$

where $\mathcal{N}[u(x)]$ is a differential operator, $u(x)$ is the problem solution, and $\{\mathcal{B}_k[\cdot]\}$ is a set of $K$ boundary conditions.

PINNs use neural networks $u(x, \boldsymbol{\theta})$ to approximate solutions, where $\boldsymbol{\theta}$ represents network parameters. They are trained using a loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_I \sum_{i=1}^{N_I} \|\mathcal{N}[u(x_i, \boldsymbol{\theta})] - f(x_i)\|^2 + \sum_{k=1}^{K} \lambda_B^{(k)} \sum_{i=1}^{N_B^{(k)}} \|\mathcal{B}_k[u(x_i^{(k)}, \boldsymbol{\theta})] - g_k(x_i^{(k)})\|^2 \tag{3}$$

While PINNs offer advantages such as being mesh-free and easily extendable to inverse problems, they struggle with complex, multi-scale solutions due to spectral bias and super-linear growth of the optimization problem.

Finite basis physics-informed neural networks (FBPINNs) attempt to address these limitations by decomposing the domain into overlapping subdomains and placing separate neural networks in each subdomain. However,

FBPINNs remain computationally expensive compared to traditional methods due to their reliance on gradient descent-based optimization.

This paper introduces Extreme Learning Machine Finite Basis Physics-Informed Neural Networks (ELM-FBPINNs), which significantly accelerate the training of FBPINNs by linearizing their underlying optimization problem.

# 2 Methodology

## 2.1 Extreme Learning Machines (ELMs)

An ELM is a neural network where only the weights and biases of the last layer are trained; all other parameters remain fixed as initialized. For a shallow network approximating a function $g(x)$, the ELM approximant is:

$$u_{ELM}(x, \mathbf{a}) = \sum_{c=1}^{C} a_c \sigma(\mathbf{w}_c \cdot x + b_c), \tag{4}$$

where $\mathbf{a} = [a_1, \ldots, a_C]^T$ contains the trainable weights of the last layer, while $\mathbf{w}_c$ and $b_c$ are randomly initialized and fixed. Given collocation points $\{x_i\}_{i=1}^N$, the ELM is trained by minimizing:

$$\mathcal{L}(\mathbf{a}) = \frac{1}{2} \sum_{i=1}^{N} \|u_{ELM}(x_i, \mathbf{a}) - g(x_i)\|^2, \tag{5}$$

which is equivalent to solving the least-squares problem:

$$\min_{\mathbf{a} \in \mathbb{R}^C} \frac{1}{2} \|M\mathbf{a} - \mathbf{b}\|_2^2, \tag{6}$$

where $M$ is the matrix of basis function evaluations at collocation points and $\mathbf{b}$ contains the function values at those points.

The ELM algorithm consists of three steps:

1. Randomly assign input weights $\mathbf{w}_c$ and biases $b_c$ for $c = 1, \ldots, C$.
2. Calculate the hidden layer output matrix $M$.
3. Calculate the output weight vector $\mathbf{a}$ by solving the least-squares problem.

## 2.2 ELM-FBPINN Method

The ELM-FBPINN method replaces the subdomain networks in FBPINNs with ELMs to solve PDEs. The domain $\Omega$ is decomposed into $J$ overlapping subdomains $\Omega = \cup_{j=1}^{J} \Omega_j$, with an ELM placed in each subdomain. The global solution is given by:

$$u(x, \mathbf{a}) = \sum_{j=1}^{J} \omega_j(x) \sum_{c=1}^{C} a_{j,c} \sigma(\mathbf{w}_{j,c} \cdot x + b_{j,c}), \tag{7}$$

where $\omega_j(x)$ are window functions that confine each subdomain network to its subdomain and typically obey a partition of unity ($\sum_{j=1}^{J} \omega_j \equiv 1$ on $\Omega$).

The ELM-FBPINN is trained using the loss function:

$$\mathcal{L}(\mathbf{a}) = \frac{1}{2} \sum_{i=1}^{N_I} \lambda_I^{(i)} \|\mathcal{N}[u(x_i, \mathbf{a})] - f(x_i)\|^2 + \frac{1}{4} \sum_{k=1}^{K} \sum_{i=1}^{N_B^{(k)}} \lambda_B^{(i,k)} \|\mathcal{B}_k[u(x_i^{(k)}, \mathbf{a})] - g_k(x_i^{(k)})\|^2 \tag{8}$$

Assuming that $\mathcal{N}$ and $\mathcal{B}$ are linear operators, this can be reformulated as a least-squares problem:

$$\min_{\mathbf{a} \in \mathbb{R}^{CJ}} \frac{1}{2} \|D_I(M\mathbf{a} - \mathbf{c})\|_2^2 + \frac{1}{4} \|D_B(B\mathbf{a} - \mathbf{g})\|_2^2, \tag{9}$$

where:

- $\mathbf{c} = \{f(x_n)\}_{n=1}^{N_I}$ contains evaluations of the PDE right-hand side
- $\mathbf{g} = \{g_p\}_{p=1}^{N_B}$ contains evaluations of boundary condition right-hand sides
- $\mathbf{a} = \{a_\ell\}_{\ell=1}^{JC}$ contains the weights of all subdomain ELMs
- $M = \{m_{n,\ell}\}$ where $m_{n,\ell} = \mathcal{N}[(\omega_j \Psi_{j,c})(x_n)]$
- $B = \{b_{p,\ell}\}$ where $b_{p,\ell} = \mathcal{B}_k[(\omega_j \Psi_{j,c})(x_i^{(k)})]$
- $D_I$ and $D_B$ are scaling matrices that ensure the maximum element in each row of $D_I M$ and $D_B B$ has magnitude 1

The window functions make the matrices $M$ and $B$ sparse, which can be exploited for computational efficiency.

## 3 Theoretical Properties

The ELM-FBPINN approach offers several theoretical advantages:

1. **Linearization of Optimization**: By using ELMs as subdomain networks, the non-convex optimization problem in traditional FBPINNs is transformed into a linear least-squares problem, which can be solved much more efficiently.

2. **Bounded Condition Number**: Numerical evidence suggests that the condition number of the matrix in the least-squares problem remains relatively stable as the number of subdomains increases, indicating that larger problems do not become significantly more challenging to solve.

3. **Domain Decomposition Benefits**: The domain decomposition approach helps mitigate the spectral bias problem that affects standard PINNs, allowing the method to better capture high-frequency components of the solution.

The method combines the approximation capabilities of FBPINNs with the computational efficiency of solving linear systems.

## 4 Numerical Examples and Results

The paper demonstrates the ELM-FBPINN method on a one-dimensional damped harmonic oscillator problem:

$$m\frac{d^2u}{dt^2} + \mu\frac{du}{dt} + ku(t) = 0, \quad t \in (0, 1], \tag{10}$$

$$u(0) = 1, \tag{11}$$

$$\frac{du}{dt}(0) = 0, \tag{12}$$

where $m$ is the mass, $\mu$ is the friction coefficient, and $k$ is the spring constant. The under-damped case is considered, with $m = 1$, $\omega_0 = 80$, and $\delta = 2$, where $\delta = \frac{\mu}{2m}$ and $\omega_0 = \sqrt{\frac{k}{m}}$.

The exact solution for this problem is $u_{exact}(t) = e^{-\delta t}(2A\cos(\phi + \omega t))$ with $\omega = \sqrt{\omega_0^2 - \delta^2}$, $A = \frac{1}{2\cos(\phi)}$, and $\phi = \arctan(-\frac{\delta}{\omega})$.

The experimental setup included:

- 150 collocation points for training
- 300 evenly spaced test points on $[0, 1]$
- 20 equally spaced subdomains with width 0.19 for ELM-FBPINN and FBPINN
- 32 neurons per subdomain for FBPINN
- 32 basis functions per subdomain for ELM-FBPINN
- Two hidden layers with 64 neurons each for PINN
- Sine activation functions for all models
- Adam optimizer with learning rate 0.001 for 20,000 training steps for FBPINN and PINN

The results demonstrate that:

1. **Accuracy Comparison**: Both FBPINN and ELM-FBPINN approximated the solution well, with final $L^1$ losses of 0.00311 and 0.00587, respectively. The standard PINN performed poorly (final loss 0.226) due to spectral bias affecting its ability to capture high-frequency components.
2. **Convergence Time**: The ELM-FBPINN achieved a solution with precision comparable to the FBPINN but in a fraction of the time. While the FBPINN required many iterations of gradient descent, the ELM-FBPINN only needed to solve a linear system once.
3. **Condition Number Analysis**: As the number of subdomains increased from 5 to 25, the condition number of the ELM-FBPINN matrix remained relatively stable. This suggests that the method can scale to larger problems without significant degradation in the conditioning of the linear system.

# 5 Implications and Advantages

The ELM-FBPINN method offers several significant advantages:

1. **Computational Efficiency**: By transforming the non-convex optimization problem into a linear least-squares problem, ELM-FBPINN achieves orders of magnitude faster training times compared to standard FBPINNs.

2. **Comparable Accuracy**: Despite the simplification of using ELMs with fixed random parameters in the hidden layers, the method maintains accuracy comparable to fully-trained FBPINNs.

3. **Scalability**: The stable condition number as the number of subdomains increases suggests good scalability to larger problems.

4. **Mitigation of Spectral Bias**: The domain decomposition approach helps address the spectral bias issue that plagues standard PINNs, allowing the method to better capture high-frequency components of solutions.

5. **Matrix Sparsity**: The window functions introduce sparsity in the system matrices, which can be exploited for computational efficiency, particularly in higher-dimensional problems.

The ELM-FBPINN approach represents a significant step toward making physics-informed neural networks computationally competitive with traditional numerical methods while maintaining their flexibility and accuracy.

Future research directions include:

- Investigating optimal strategies for initializing the untrained weights and biases in the ELM
- Extending the method to more challenging problems in higher dimensions
- Exploring adaptive domain decomposition strategies
- Developing efficient preconditioners for the resulting linear systems